TrOOPL specification

Version 0.1

TrOOPL is a minimal object-oriented programming language with a purely functional core, where dependency injection, mutability, and capabilities like I/O and file system access are managed through explicitly passed and constructed environment objects.

Contents

1. Lexical structure 3
   - 1.1. Source text 3
   - 1.2. Comments 3
   - 1.3. String literals 3
   - 1.4. Identifiers 3
     - 1.4.1. Numeric literals 3
   - 1.5. Punctuation 3
   - 1.6. Whitespace 3
   - 1.7. Tokenization rules 3
2. Grammar 5
   - 2.1. Programs 5
   - 2.2. Sequences of statements 5
   - 2.3. Non-terminating statements 5
   - 2.4. Terminating statements 5
   - 2.5. Definitions 5
   - 2.6. Recursive definitions 5
   - 2.7. Non-recursive definitions 5
   - 2.8. Object-like values 6
   - 2.9. Non-object-like values 6
   - 2.10. Values 6
   - 2.11. Objects 6
   - 2.12. Object entries 6
   - 2.13. Method definitions 6
   - 2.14. Blocks 6
   - 2.15. Method call chains 7
   - 2.16. Environment call chains 7
   - 2.17. Full grammar 7
3. Abstract syntax 8
   - 3.1. Expressions 8
   - 3.2. Object entries 9
   - 3.3. Blocks 9
   - 3.4. Terminating definitions 9
4. Static semantics 11
   - 4.1. Scope 11
5. Dynamic semantics 12
   - 5.1. Definitions 12
   - 5.2. Recursive definitions 12
   - 5.3. Sequences 12
   - 5.4. Object expressions 12
   - 5.5. Variables 13
   - 5.6. Numbers 13
   - 5.7. String literals 13
   - 5.8. Environments 13
   - 5.9. Method calls 13
   - 5.10. Tail call optimization 13
   - 5.11. Objects 13
   - 5.12. Errors 14

6. Builtins 15
6.1. Booleans 15
6.2. Numbers 15
6.3. Strings 16
6.4. Lists 16
6.5. Sequences 17
6.6. Errors 17
6.7. Environment 17

1. Lexical structure

This section describes how a TrOOPL program is split into tokens during compilation.

1.1. Source text

A TrOOPL source is a sequence of UTF-8 characters.

The following characters are recognized as special characters:

```
( ) { } [ ] # "
```

Listing 1: Special characters

1.2. Comments

`#`, followed by any sequence of characters till the end of the line is a comment, which is ignored during parsing:

```
# This is a comment
```

Listing 2: Comment example

1.3. String literals

`"`, followed by a sequence of non-`"` characters and `""` pairs, up to and including a terminating `"`, is a string literal. A `""` pair is used to escape a single `"`.

Carriage returns in a string literals are ignored, to not force line ending to be either CRLF or LF. If you want a string with a carriage return character, construct it manually.

```
"This is a string literal"
```

Listing 3: String literal example

1.4. Identifiers

An identifier is any sequence of non-special and non-whitespace characters. TrOOPL has identifiers of two kinds: variable names and method names.

Variable names are identifiers that start with an uppercase letter.

Numeric literals are identifiers that start with a digit (Not all such identifiers are valid numbers, however).

Method names are identifiers that start with anything other than a digit or an uppercase letter.

```
Variable-Name
123.5-
method-name
```

Listing 4: Identifiers example

1.4.1. Numeric literals

Not every identifier starting with a digit is a valid numeric literal. A valid numeric literal consists of only digits, with a single optional `.` anywhere except as first character, and then an optional `-` at the end. The `.` is a decimal point. The `-` at the end means the number is negative. The reason it goes at the end and not at the beginning is for consistency with the rest of the syntax.

1.5. Punctuation

`(`, `)`, `[`, `]`, `{`, and `}` are all their own individual tokens.

1.6. Whitespace

Whitespace between tokens is ignored during parsing.

1.7. Tokenization rules

The precise rules for tokenization of TrOOPL are defined using these greedy regular expressions:

```
<variable>   ::= [\p{Lu}\p{Lt}][^\(\)\[\]\{\}"# \t\r\n]*
<number>     ::= [0-9][^\(\)\[\]\{\}"# \t\r\n]*
<method>     ::= [^0-9\p{Lu}\p{Lt}\(\)\[\]\{\}"# \t\r\n][^\(\)\[\]\{\}"# \t\r\n]*
<string>     ::= "([^"]|"")*"
<comment>    ::= #[^\n]*
<whitespace> ::= [ \t\r\n]+
```

Listing 5: Tokenization rules

2. Grammar

This section describes the concrete syntax of TrOOPL.

2.1. Programs

A TrOOPL program is a sequence of statements. For example, see Listing 6.

2.2. Sequences of statements

A sequence of statements consists of a sequence of non-terminating statements, followed by a terminating statement, or it is a call chain on the environment object.

```
A 1
B 2
A + B
```

```
io print "Hello World"
```

```
# empty sequence evaluates to current environment
```

Listing 6: Examples of a sequences of statements

2.3. Non-terminating statements

A non-terminating statement is either a variable definition, or an arbitrary value (But not a variable, as that would make variable definitions ambiguous).

```
Var-A Value
```

```
Var-B (1 + 1)
```

```
(Logger log "Hello")
```

Listing 7: Examples of non-terminating statements

2.4. Terminating statements

A terminating statement is either a variable definition, or a method call chain.

```
Var Value
```

```
1 + 1 = 2 then-else "Yes" "No"
```

Listing 8: Examples of terminating statements

2.5. Definitions

A variable definition is either a recursive definition, or a non-recursive definition. For examples, see Listing 9 and Listing 10.

2.6. Recursive definitions

A recursive definition consists of a variable name, followed by an object-like value.

```
Inf-Recur [Inf-Recur run]
```

```
Var {self (Var)}
```

Listing 9: Examples of recursive definitions

2.7. Non-recursive definitions

A non-recursive definition consists of a variable name, followed by non-object-like value.

```
Var-A (Var-A + 1) # creates a new Var-A from old Var-A
```

```
Var-B "string"
```

```
Var-C Var-C # no-op
```

Listing 10: Examples of non-recursive definitions

2.8. Object-like values

An object-like value is either an object, or a block.

```
{method "value"}
```

```
[1 + 1]
```

Listing 11: Examples of object-like values

2.9. Non-object-like values

A non-object-like value is a variable, a numeric literal, a string literal, or a parenthesized statement list.

```
Variable
```

```
123.5-
```

```
"string"
```

```
(1 + 1)
```

```
()
```

Listing 12: Examples of non-object-like values

2.10. Values

A value is either an object-like or a non-object-like value. For examples, see Listing 11 and Listing 12.

2.11. Objects

An object is a sequence of object entries surrounded by curly braces.

```
{Super-Object method-a A B (A + B) method-b "" (Object get-super-object)}
```

Listing 13: Example of an object

2.12. Object entries

An object entry is either a method definition, or an included value.

```
Super-Object
```

```
(Object get-super-object)
```

```
method A B (A + B)
```

Listing 14: Examples of object entries

2.13. Method definitions

A method definition consists of a method name, followed by a sequence of variables names (Argument names), terminated by the body, which is a value (But not a variable, as that could be ambiguously parsed as another argument name).

```
sum A B (A + B)
```

```
get-a (A)
```

Listing 15: Examples of method definitions

2.14. Blocks

A block consists of a sequence of statements, surrounded by square brackets.

```
[
  A 1
  B 2
  A + B
]
```

Listing 16: Example of a block

2.15. Method call chains

A single method call consists of a value, followed by a method name, followed by the arguments (A value list). A method call chain is formed by chaining method calls, left-associatively (With possibly zero calls, in which case it's just a value).

```
Object method-a method-b Argument-A method-c (1 + 1) Argument-C
```

```
Value
```

Listing 17: Examples of method call chains

2.16. Environment call chains

When a sequence of statements starts with a method, it's an environment call chain. The syntax is similar to Section 2.15, with the only difference being that there is no starting value at the start.

```
io print "Hello World"
```

```
# empty chain, returns environment itself
```

Listing 18: Examples of environment call chains

2.17. Full grammar

The full grammar can be described as the following token-level Backus-Naur form:

```
<program>                     ::= <full-statements>
<full-statements>             ::= <statements> | <env-call>
<statements>                  ::= <non-terminating-statement> <statements>
                                | <terminating-statement>
<non-terminating-statement>   ::= <definition> | <value-except-variable>
<terminating-statement>       ::= <definition> | <call>
<definition>                  ::= <recursive-definition>
                                | <non-recursive-definition>
<recursive-definition>        ::= <variable> <object-like>
<non-recursive-definition>    ::= <variable> <non-object-like>
<object-like>                 ::= <object> | <block>
<non-object-like>             ::= <variable> | <not-object-like-or-variable>
<not-object-like-or-variable> ::= <number> | <string> | "(" <full-statements> ")"
<value>                       ::= <object-like> | <non-object-like>
<value-except-variable>       ::= <object-like> | <not-object-like-or-variable>
<object>                      ::= "{" <object-entries> "}"
<object-entries>              ::= "" | <object-entries> <object-entry>
<object-entry>                ::= <method-definition> | <value>
<method-definition>           ::= <method> <arguments> <value-except-variable>
<arguments>                   ::= "" | <arguments> <variable>
<block>                       ::= "[" <full-statements> "]"
<call>                        ::= <value> | <call> <method> <values>
<values>                      ::= "" | <values> <value>
<env-call>                    ::= "" | <env-call> <method> <values>
```

Listing 19: Full BNF grammar

3. Abstract syntax

This section describes the abstract syntax of TrOOPL.

3.1. Expressions

Expression can be represented as the following data structure (Using Haskell-like syntax):

```
data Expression =
    Definition {name :: String, value :: Expression, scope :: Expression}
  | RecursiveDefinition {
      name    :: String,
      entries :: List ObjectEntry,
      scope   :: Expression}
  | Sequence {first :: Expression, second :: Expression}
  | Object {entries :: List ObjectEntry}
  | Variable {name :: String}
  | NumericLiteral {value :: Number}
  | StringLiteral {value :: String}
  | Environment
  | Call {
      object    :: Expression,
      method    :: String,
      arguments :: List Expression}
```

Listing 20: Definition of an expression

For instance, the concrete syntax:

```
A (0 1 + 2)
B {self B}
B ({A ignore-x-and-return-b X B})
[custom-print ("Hello" + " " + "World")]
```

... corresponds to the following abstract syntax:

```
Definition {
  name  = "A",
  value = Sequence {
    first  = NumericLiteral {value = 0},
    second = Call {
      object    = NumericLiteral {value = 1},
      method    = "+",
      arguments = [NumericLiteral {value = 2}]}},
  scope = RecursiveDefinition {
    name    = "B",
    entries = [MethodDefinition {
      name      = "self",
      arguments = [],
      body      = Variable {name = "B"}}],
    scope   = Definition {
      name  = "B",
      value = Object {entries = [
        Inclusion {value = Variable {name = "A"}},
        MethodDefinition {
          name      = "ignore-x-and-return-b",
          arguments = ["X"],
          body      = Variable {name = "B"}}]},
      scope = Object {entries = [MethodDefinition {
        name      = "run",
        arguments = [],
        body      = Call {
          object    = Environment,
          method    = "custom-print",
          arguments = [Call {
            object    = Call {
              object    = StringLiteral {value = "Hello"},
              method    = "+",
              arguments = [StringLiteral {value = " "}]},
            method    = "+",
            arguments = [StringLiteral {value = "World"}]}]}]}]}}}
```

Listing 21: Example of an abstract syntax tree

3.2. Object entries

Object entries can be represented as the following data structure:

```
data ObjectEntry =
    MethodDefinition {
      name      :: String,
      arguments :: List String,
      body      :: Expression}
  | Inclusion {value :: Expression}
```

Listing 22: Definition of an object entry

For examples of object entries, refer to Listing 21.

3.3. Blocks

A block `[...]` is syntactic sugar for an object `{run (...)}`.

For an example of desugared abstract syntax tree representation of a block, see Listing 21.

3.4. Terminating definitions

A variable definition `Var Value` in the terminating position of a sequence of statements is equivalent to `Var Value Var`.

Therefore, it gets transformed into the abstract syntax tree as follows:

- If the definition is a non-recursive one, then it results in the following structure:

```
Definition {
  name  = name,
  value = value,
  scope = Variable {name = name}}
```

- If the definition is a recursive one, then it results in the following structure:

```
RecursiveDefinition {
  name    = name,
  entries = entries,
  scope   = Variable {name = name}}
```

4. Static semantics

This section describes the static semantics of TrOOPL.

4.1. Scope

TrOOPL uses lexical scoping for variables.

A definition:

```
Definition {name :: String, value :: Expression, scope :: Expression}
```

... brings the variable binding named `name` into the scope of the expression `scope`, shadowing any existing bindings of the same name.

A recursive definition:

```
RecursiveDefinition {
  name    :: String,
  entries :: List ObjectEntry,
  scope   :: Expression}
```

... in addition to bringing the variable binding into the scope of the expression `scope`, also brings it to the scope of expressions in `entries`.

If the program references variables that were not explicitly bound, a compile-time error is issued.

5. Dynamic semantics

This section describes the runtime behavior of a valid TrOOPL program.

5.1. Definitions

A definition:

```
Definition {name :: String, value :: Expression, scope :: Expression}
```

... when evaluated:

1. Evaluates `value`;
2. Puts its result into the variable slot corresponding to the new variable named `name`;
3. Evaluates the expression `scope` and returns its result.

5.2. Recursive definitions

A recursive definition:

```
RecursiveDefinition {
  name    :: String,
  entries :: List ObjectEntry,
  scope   :: Expression}
```

... when evaluated:

1. Creates an empty object, marks it as uninitialized, and puts it into the variable slot corresponding to the new variable named `name`;
2. Iterates through each object entry, and for each one:
   - If it's a method definition, adds it to the object being constructed, replacing a method with same combination of name + arity if such method already exists;
   - If it's object inclusion, evaluates the included expression, and adds all of the methods of the resulting object to the one being constructed, replacing methods with name + arity combinations that were already there. Also, if the included object has internal numeric representation, adds that numeric representation to the constructed object, or replaces it if the constructed object already has one;
3. Marks the object as initialized;
4. Evaluates the expression `scope` and returns its result.

5.3. Sequences

A sequence:

```
Sequence {first :: Expression, second :: Expression}
```

... when evaluated, first evaluates `first`, then evaluates `second` and returns its result.

5.4. Object expressions

An object expression:

```
Object {entries :: List ObjectEntry}
```

... when evaluated:

1. Creates an empty object and marks it as uninitialized;
2. Iterates through each object entry, and for each one:
   - If it's a method definition, adds it to the object being constructed, replacing a method with same combination of name + arity if such method already exists;
   - If it's object inclusion, evaluates the included expression, and adds all of the methods of the resulting object to the one being constructed, replacing methods with name + arity combinations that were already there. Also, if the included object has internal numeric representation, adds that numeric representation to the constructed object, or replaces it if the constructed object already has one. Trying to include an uninitialized object will result in an error;
3. Marks the object as initialized and returns it as the result.

5.5. Variables

A variable expression:

```
Variable {name :: String}
```

... when evaluated, returns the object stored at the slot corresponding to the variable named `name` in the current scope.

5.6. Numbers

A numeric literal expression:

```
NumericLiteral {value :: Number}
```

... when evaluated, returns a number object as described in Section 6.2, that represents the number `value`.

5.7. String literals

A string literal expression:

```
StringLiteral {value :: String}
```

... when evaluated, returns a string object as described in Section 6.3, that represents the string `value`.

5.8. Environments

An environment expression:

```
Environment
```

... when evaluated, returns the current dynamic environment. See Section 6.7

5.9. Method calls

A method call:

```
Call {object :: Expression, method :: String, arguments :: List Expression}
```

... when evaluated:

1. Evaluates the expression `object`, if the returned object is marked as uninitialized, aborts and finishes the program with failure;
2. Evaluates each expression in `arguments` list, from left to right;
3. Calls the method named `method` on the object returned by the expression `object`, with objects returned from expressions from `arguments` list as the arguments, and returns the result of that method call, aborting and finishing the program with failure if the object doesn't have a method of such name + arity.

5.10. Tail call optimization

An expression is in tail call position if either:

- It's the whole body of a method;
- It's the `scope` field of a `Definition` expression that is in tail call position;
- It's the `second` field of a `Sequence` expression that is in tail call position.

A `Call` expression in tail call position is guaranteed to reuse the stack frame of the method it is located in, allowing to encode arbitrarily long loops using recursion. Without this optimization, trying to do so might result in a stack overflow.

5.11. Objects

Every value in TrOOPL is an object. An object is a set of methods, where each method is just a closure. Objects are passed by reference.

Unlike in other object-oriented languages, methods in TrOOPL don't accept an implicit "self" argument. Instead, you are expected to use recursive definitions to achieve the same functionality.

5.12. Errors

When an error occurs, program terminates with no ability to recover. Recoverable errors are encoded with values.

When reporting an error, the interpreter gets the filename by calling `filename` on the current environment. If the method does not exist, or produces an invalid string object, then an implementation-defined default will be used.

6. Builtins

This section lists what builtins are available to a TrOOPL program, as well as describes their interfaces.

Builtins are objects that are either produced or expected by builtin methods, or provided by the interpreter directly.

6.1. Booleans

TrOOPL provides two default environment methods for booleans: `true` and `false`, returning boolean objects representing true and false values, respectively.

A boolean is an object with a method `then-else X Y`, which returns `X` if the boolean object represents a true value, and `Y` if the boolean object represents a false value.

Standard boolean operations such as `and`, `or`, and `not` can all be implemented in terms of `then-else`.

Builtin methods that consume booleans as their arguments do not use any internal representation or method of the argument other than the `then-else` method.

When a boolean is constructed by the interpreter (Or by `true` or `false` methods), it is not what the user actually receives. The minimal boolean is first passed into `core extend boolean` as its only argument, with result of that function being returned to the user.

6.2. Numbers

The interpreter creates numbers whenever it evaluates a numeric literal.

When the interpreter creates a number, it creates an object with some internal numeric representation (The exact representation is unspecified, but the prototype uses OCaml's `float` type) so that methods that consume numbers as arguments can access the value those object represent, not only their interface.

This representation is not visible to the programmer directly, and only affects the behavior of builtin methods that accept numbers.

When a builtin expecting a number is called with an object that does not have such internal numeric representation attached, the program terminates with a failure.

Builtin methods consuming numbers as arguments do not depend on the object's methods, and only care about the internal numeric representation.

In addition to the internal numeric representation, numbers have following methods (The number object the methods are called on is referred to as `Self`):

- `+ X` — produces a new number object that represents the sum of numbers represented by `Self` and `X`;
- `- X` — produces a new number object that represents the difference of numbers represented by `Self` and `X`;
- `-` — produces a new number object that represents the negation of the number represented by `Self`;
- `* X` — produces a new number object that represents the product of numbers represented by `Self` and `X`;
- `/ X` — produces a new number object that represents the result of division of the number represented by `Self`, by the number represented by `X`. The behavior on division by 0 is unspecified, as it may vary for different internal representations. In case of the prototype, the behavior is that of Java's `Double` type;
- `mod X` — produces a new number object that represents the remainder of Euclidean division of the number represented by `Self`, by the number represented by `X`. The behavior on division by 0 is unspecified;
- `rem X` — produces a new number object that represents the signed remainder of division of the number represented by `Self`, by the number represented by `X`. The behavior on division by 0 is unspecified;
- `^ X` — produces a new number object that represents the result of exponentiation of the number represented by `Self`, to the power of the number represented by `X`. Raising negative numbers to integer powers must work without failures. The result of raising negative numbers to non-integer powers is unspecified;

- `= X` — produces a boolean object that shows whether the number represented by `Self` is equal to the number represented by `X`;
- `<> X` — produces a boolean object that shows whether the number represented by `Self` is not equal to the number represented by `X`;
- `< X` — produces a boolean object that shows whether the number represented by `Self` is strictly less than the number represented by `X`;
- `> X` — produces a boolean object that shows whether the number represented by `Self` is strictly greater than the number represented by `X`;
- `<= X` — produces a boolean object that shows whether the number represented by `Self` is less than or equal to the number represented by `X`;
- `>= X` — produces a boolean object that shows whether the number represented by `Self` is greater than or equal to the number represented by `X`;
- `floor` — produces a new number object that represents the number represented by `Self` rounded down;
- `ceil` — produces a new number object that represents the number represented by `Self` rounded up;
- `trunc` — produces a new number object that represents the number represented by `Self` rounded towards zero;
- `to-string` — produces a string object containing the decimal representation of the number represented by `Self`. The precision of the decimal representation is unspecified, but the method must produce a string that counts as a numeric literal according to Section 1.4.1, if possible. Not all numbers are possible to represent this way, for instance infinities and NaN. For them, the representation is unspecified.

When a number is constructed by the interpreter, instead of being immediately returned to the user, it first gets passed into `core extend number` as its only argument, with the result of that call being returned to the user.

6.3. Strings

The interpreter creates string objects whenever it evaluates a string literal. It immediately constructs the number objects for each index from left to right, then constructs number object representing length.

A string is an object with following methods:

- `at X` — returns the character of the string at index `X` (Zero-indexed) as a number object representing the Unicode code point. Default implementation returns the exact same number object every time you call this method with the same index;
- `size` — returns the size of the string in Unicode code points. Default implementation returns the exact same number object every time you call this method.

Builtin methods that consume strings as their arguments do not use any internal representation or methods of the argument other than those two methods.

When a string is constructed by the interpreter, instead of being immediately returned to the user, it first gets passed into `core extend string` as its only argument, with the result of that call being returned to the user.

6.4. Lists

Lists are constructed and consumed by some of the methods from the default environment. If has the following methods:

- `at X` — returns element at index `X` (Zero-indexed);
- `size` — returns the size of the list.

Builtin methods that produce lists materialize the lists ahead of time, so call to both of the above methods return same instances given you call them with same arguments. This is not a general requirement for user-created lists.

As you may have noticed, strings are a subset of lists, and therefore can be passed to functions expecting lists.

When default environment's methods create lists, they are not returned directly, but passed into `core extend list`, result of which is then returned.

6.5. Sequences

Sequences are not produced by the interpreter or default environment's methods, but are consumed by `core from-seq`. Sequences represent delayed linked lists, and work similar to OCaml's sequences. A sequence has just one method:

- `run` — computes the sequence node.

A sequence node has the following methods:

- `cons?` — returns a boolean. if true, the node also has the following two methods, if false, it represents a NIL value and therefore does not have those methods;
- `get` — gets value stored in this sequence node;
- `next` — gets the next sequence in the chain.

6.6. Errors

Recoverable errors are returned as values. They are extensions of other types, with extra `ok?` method attached, returning a boolean.

If `ok?` returns true, the rest of the method set is that of a normal type.

If `ok?` returns false, the rest of the method set does not contain methods of the extended type, but rather error information. For custom errors, programmer is free to choose any kinds of methods for that, but default environment uses `message` method, returning a string with description of the error.

6.7. Environment

TrOOPL provides a special builtin value called the environment. An environment is what by default contains all of the machinery required to introduce impure computations into the language, such as creating mutable state, I/O, and file system access.

Unlike variables, environment is dynamically scoped, meaning instead of inheriting the environment of surrounding lexical scope, environment is inherited from the caller of the method currently being evaluated.

When the program starts, the environment has the following interface:

```
Default-Environment {
  io { # let's call this `io`; operations are UTF-8
    print     X (...) # X: string -> io? = prints X, with newline, returns `io`
    raw-print X (...) # X: string -> io? = prints X, no newline/flush, returns `io`
    read-line   (...) # -> string? = reads one line from stdin, returns it with LF stripped
    flush       (...) # -> io? = flush stdout
    error { # let's call this `err`
      print     X (...) # X: string -> err? = print X to stderr, with LF, return `err`
      raw-print X (...) # X: string -> err? = print X to stderr, no LF, return `err`
      flush       (...) # -> err? = flush stderr
    }
  }
  fs { # let's call this `fs`
    read  F   (...) # F: string -> string? = read whole file F as string (no CRLF normalization)
    write F S (...) # F: string, S: string -> fs? = write string in S to file F, return `fs`
    open-read F { # F: string -> read-file? = opens file F for reading
      read N (...) # N: int -> list? = read at most N bytes (N > 0). If empty, it's EOF
      close  (...) # -> {}? = close file, return empty object on success
    }
    open-write F { # F: string -> write-file? = opens file F for writing
      write X (...) # X: list -> write-file? = writes bytes X to file, returns self
      close   (...) # -> {}? = close file, return empty object on success
    }
    metadata F { # F: string -> metadata? = return metadata for file at path F (stat)
      exists?  (...) # -> bool = true if file exists
      file?    (...) # -> bool = true if file exists and is a normal file
      dir?     (...) # -> bool = true if file exists and is a directory
      other?   (...) # -> bool = true if file exists and is of any other type than above
      accessed (...) # -> num = access time as Unix timestamp (0 if file doesn't exit)
      modified (...) # -> num = modification time as Unix timestamp (0 if file doesn't exit)
      size     (...) # -> int = total file size in bytes (0 if file doesn't exit)
    }
    real-path   F (...) # F: string -> string? = absolute path to F, without ./.. and symlinks
    rename   F F' (...) # F: string, F': string -> fs? = rename file F to F'
    make-dir    D (...) # D: string -> fs? = create directory named D
    remove      F (...) # F: string -> fs? = remove file F
    remove-dir  D (...) # D: string -> fs? = remove empty directory D
    list-dir    D (...) # D: string -> list? = list files in a directory (list of strings)
    working-dir   (...) # -> string? = return current working directory
    working-dir D (...) # D: string -> fs? = set current working directory
  }
  system {
    run C (...) # C: list -> command-result? = equivalent to `system run C ""`
                # does not actually construct the empty string
    run C I { # C: list, I: string -> command-result? = run command C with I as stdin
      stdout    (...) # -> string = captured stdout
      stderr    (...) # -> string = captured stderr
      exit-code (...) # -> int = exit code
    }
    time (...) # -> num = Unix timestamp
    exit N (...) # N: int -> never = terminate the program with exit code N
  }
```

```
  utf-8 {
    to-string X (...) # X: list -> string+{remainder}?{message,index,byte} =
                      # convert bytes X to string, without taking negatives into account
                      # on success/incomplete input, returns a string with extra method:
                      #   remainder: returns a list of bytes unparsed so far
                      #     umaterialized: delegates to X
                      # (the string is passed to `core extend string` before adding remainder)
                      # on invalid input, returns the usual error type, with extra methods:
                      #   index: returns position of the first byte in invalid sequence
                      #   byte: returns the first byte in invalid sequence
    from-string X (...) # X: string -> list = convert string X to byte list
                        # does not take negatives into account (crashes instead)
    to-bytes   X (...) # X: string -> list = convert string X to byte list, with negatives
    from-bytes X (...) # X: list -> string = convert byte list X to string, with negatives
  }
  env {
    run  E B (...) # E: any, B: block -> type of (B run) = run B in environment E
    bind E O (...) # E: any, O: any -> type of O = wrap all methods of O in `run E [...]`
  }
  eval S (...) # S: string -> any = evaluate string S as TrOOPL code
               #                   uses current environment, fresh lexical scope
  box X {        # X: any -> box = return new box with X as content
    set X' (...) # X': any -> box = set box content to X', return self
    get    (...) # -> any = return box contents
  }
  array { # -> array = return new mutable array
    set I X (...) # I: int, X: any -> array = set value at index I to X, return self
    at  I   (...) # I: int -> any = return value at index I
    push X  (...) # X: any -> array = add X to end of array, return self
    pop     (...) # -> any = remove last element of array and return it
    size    (...) # -> int = return current size (constructs new number every time)
  }
  true  (core extend boolean {then-else A B (A)})
  false (core extend boolean {then-else A B (B)})
  core {
    extend {
      boolean B (B) # B: bool -> new-bool (called whenever new boolean is created)
      number  N (N) # N: num -> new-num (called whenever a new number is created)
      string  S (S) # S: string -> new-string (called whenever a new string is created)
      list    L (L) # L: list -> new-list (called whenever a new list is created)
    }
    tag N (...) # N: num -> num-minimal = return an object with no methods but same numeric tag
    from-seq S (...) # S: seq -> list = return a new list with values from S
  }
  filename (...) # -> string = name of the current file, used for error reporting
  crash S (...) # S: string -> never = terminate the program with error message S
}
```

... where `<type>?<error>` corresponds to an error type (See Section 6.6), extending `<type>` on success, and having methods of `<error>` on error. Omitted `<error>` implies that it is `{message}`, where `message` is a method returning a string, which is the description of the error.

Wherever a string is expected according to the types, but system doesn't guarantee UTF-8 results (Such as stdout, stderr, filenames, file contents), invalid bytes are left in as negative numbers (Since invalid bytes can only be in range [128; 255], that means such values will only be in range [−255; −128]), unless stated otherwise. Likewise, any system operation that expects a string, when converting it to bytes, must take these negative numbers into account (The whole [−255;-1] range), unless explicitly stated otherwise. This keeps string operations convenient, while still allowing to work on raw bytes if needed.